

REAL-TIME RAILWAY TRACK MONITORING SYSTEM USING OPENCV

1.RAILWAY TRACK MONITORING SYSTEM – WRITE-UP

The implementation of the Real-Time Railway Track Monitoring System consists of a computer-based vision system, whose intention is to automatically identify and detect railway track lines from photographs. Railway track monitoring is an essential safety system because any damage or abnormality in the railways may cause an accident. In this proposed system, the technology makes use of image processing and vision-based systems.

The methods utilized for implementation are Python and OpenCV. Initially, an input railway image is resized to a standard pixel size of 900 x 500 pixels for consistent processing. After resizing, a colour image is transformed into a monochrome image that helps as a means of reducing unnecessary information to intensity values.

Following pre-processing, the Canny Edge Detection technique is carried out taking lower and upper thresholds of 50 and 150 to determine the significant edges existing in the image of the railway. The edge image is processed using the Probabilistic Hough Line Transform to find out the possible railway track lines in the image. It uses an 80 threshold, 100 pixels for the minimum line length, and 30 pixels as the maximum line gapping to find out the lines.

Three primary outputs are produced, that is, the input image, Canny edge-detection image, and railway track detection image obtaining their position calculated on the original image. The detected lines appear in green with a message reading “RAILWAY TRACK DETECTED” alongside the count of lines found. The algorithm was able to detect 875 lines effectively.

Several computer vision techniques were analyzed for an advanced system with real-time monitoring. Edge + Hough Transform has the advantage of being less computationally intensive with low delays, but it lacks in accuracy. SIFT and Viola–Jones have their share of disadvantages when it comes to accuracy as well. YOLO’s performance is quite satisfactory, as it has 94 percent accuracy and 85 milliseconds latency.

On the hand deep learning is known to be much more accurate but deep learning also needs more processing power. That is why it makes sense to combine YOLO with preprocessing. By combining YOLO and OpenCV the application will perform better on edge devices.

2. TOOLS USED

Referring to your project document, the following technologies can be utilized:

- **Python** programming language is used to create the computer vision system.
- **OpenCV** is the main software library used for image processing and identification of railway tracks.
- **Canny edge detection** has been used to find significant edges in the railway track image.
- **Gaussian blur** is used to minimize noise during image processing.
- **Probabilistic Hough line transformation** is used to identify railway track lines from the identified edges.
- **NumPy** is used to manipulate images/arrays and manage line coordinates.
- **Python IDLE** is used to run the program and check the detection of the tracks.
- **YOLO** has been considered for advanced railway fault detection and intelligent system monitoring.
- **INT8 quantization** has been proposed to enhance the YOLO platform for deployment on an edge device.

3. PSEUDO CODE

3.1 Rail Feature Extraction and Track Line Identification

FUNCTION ExtractRailFeatures(image):

```
    gray <- ConvertToGray(image)
    blurred <- GaussianBlur(
        gray,
        kernel=5x5
    )
    edges <- CannyEdges(
        blurred,
        low=50,
        high=150
    )
    lines <- ProbabilisticHoughTransform(
        edges,
        rho=1,
        theta=PI/180,
```

```

        threshold=80,
        minLineLength=100,
        maxLineGap=30
    )
    detected_lines <- []
    IF lines is not empty:
        FOR each line in lines:
            points <- Reshape(line)
            IF length(points) >= 4:
                x1 <- points[0]
                y1 <- points[1]
                x2 <- points[2]
                y2 <- points[3]
                detected_lines.append(
                    (x1,y1,x2,y2)
                )
        RETURN edges, detected_lines
    END FUNCTION

```

3.2 Intelligent Railway Defect Recognition

```

FUNCTION RecognizeRailwayDefects(
    frame,
    detected_region
):
    INITIALIZE YOLO_Model
    Load edge-optimized YOLO model
    Apply INT8 quantization
    crop <- ExtractRegion(
        frame,

```

```

        detected_region
    )
    predictions <- YOLO_Model.Detect(
        crop
    )
    results <- []
    FOR each prediction:
        label <- prediction.class
        confidence <- prediction.confidence
        bbox <- prediction.bounding_box
        IF confidence >= 0.50:
            results.append(
                (
                    label,
                    bbox,
                    confidence
                )
            )
    results <- NonMaxSuppression(
        results
    )
    RETURN results
END FUNCTION

```

4. IMPLEMENTATION

4.1 Implementation Approach

The Railway Track Monitoring System being proposed employs the use of Python and OpenCV to effectively detect railway track lines from a given image. The system is based on certain image manipulation procedures applied to convert an image of the railway into an processed version with a visible railway line.

1. Image Acquisition

An image of railway track, IMAGE.png, is taken into the program using imread() method provided by OpenCV library. The input image is checked if it is successfully read in before resizing it to 900×500 pixels for uniformity.

2. Grey Image Formation

By utilizing cv2.cvtColor(), we convert the original color image into grey and black image thus simplifying our work.

3. Removing Noise

A Gaussian Blur of 5x5 is applied to the grey image in order to remove the small noise and other unnecessary variations in the image just before performing the edge detection procedure.

4. Canny Edge Detection

The blurred image is processed using the Canny edge detection algorithm with lower and upper thresholds of 50 and 150. This identifies strong edges corresponding to railway rails and other structures in the image.

5. Railway Line Detection

The detected edges are passed to the Probabilistic Hough Line Transform. The parameters used are:

- Threshold = 80
- Minimum line length = 100 pixels
- Maximum line gap = 30 pixels

The detected line coordinates are extracted safely using array reshaping.

6. Track Line Marking

Each detected line is drawn on a copy of the original railway image using a **green line**. A counter is maintained to determine the total number of detected lines.

7. Result Generation

The output image is labelled with “RAILWAY TRACK DETECTED” and the number of detected lines. The implementation displays three results:

Input Image → Canny Edge Image → Output Image

8. Performance and Result Verification

The program prints the processing method, number of detected lines, and completion status in the Python IDLE console. The generated images are displayed for visual verification of railway track localization.

4.1.1 Image Preprocessing

FUNCTION Preprocess(image):

```
    gray <- ConvertToGray(image)
```

```
    blur <- GaussianBlur(
```

```
        gray,
```

```
        kernel=5x5,
```

```
        sigma=0
```

```
    )
```

```
    RETURN gray, blur
```

4.1.2 Edge Detection and Rail Line Extraction

FUNCTION DetectRailLines(blur):

```
    edges <- CannyEdges(
```

```
        blur,
```

```
        low=50,
```

```
        high=150
```

```
    )
```

```
    lines <- ProbabilisticHoughTransform(
```

```
        edges,
```

```
        rho=1,
```

```
        theta=PI/180,
```

```
        threshold=80,
```

```
        minLineLength=100,
```

```
        maxLineGap=30
```

```
    )
```

```
    RETURN edges, lines
```

4.1.3 Railway Track Line Marking

FUNCTION MarkRailLines(image, lines):

 output <- Copy(image)

 line_count <- 0

 IF lines is not None:

 FOR each line in lines:

 points <- Reshape(line)

 IF length(points) >= 4:

 x1 <- Integer(points[0])

 y1 <- Integer(points[1])

 x2 <- Integer(points[2])

 y2 <- Integer(points[3])

 DrawLine(

 output,

 (x1,y1),

 (x2,y2),

 color=GREEN,

 thickness=3

)

 line_count <- line_count + 1

 RETURN output, line_count

4.1.4 Final Output Generation

FUNCTION GenerateOutput(image):

 gray, blur <- Preprocess(image)

 edges, lines <- DetectRailLines(

 blur

)

 output, line_count <- MarkRailLines(

```

        image,
        lines
    )
    AddText(
        output,
        "RAILWAY TRACK DETECTED"
    )
    AddText(
        output,
        "Lines Detected: line_count"
    )
    RETURN image,
        edges,
        output,
        line_count

```

4.1.5 Main Program

```

BEGIN RailwayTrackMonitoring
    INPUT:
        Railway track image "IMAGE.png"
    image <- ReadImage(
        "IMAGE.png"
    )
    IF image is None:
        Display(
            "ERROR: IMAGE.png not found!"
        )
    EXIT
    Display(

```



```
    "Image loaded successfully"
)
image <- Resize(
    image,
    width=900,
    height=500
)
input_image,
edges,
output,
line_count <- GenerateOutput(
    image
)
Display(
    "INPUT - Railway Track",
    input_image
)
Display(
    "CANNY EDGE DETECTION",
    edges
)
Display(
    "OUTPUT - Railway Track Monitoring",
    output
)
Display(
    "Method: Canny + Hough Transform"
)
Display(
```

```
"Lines Detected: line_count"  
)  
Display(  
    "Status: Detection Completed"  
)  
WAIT for key press  
CloseAllWindows  
END RailwayTrackMonitoring
```

4.2 OUTPUT SCREENSHOTS

Input Image: Shows the original railway track image used as input for the monitoring system.

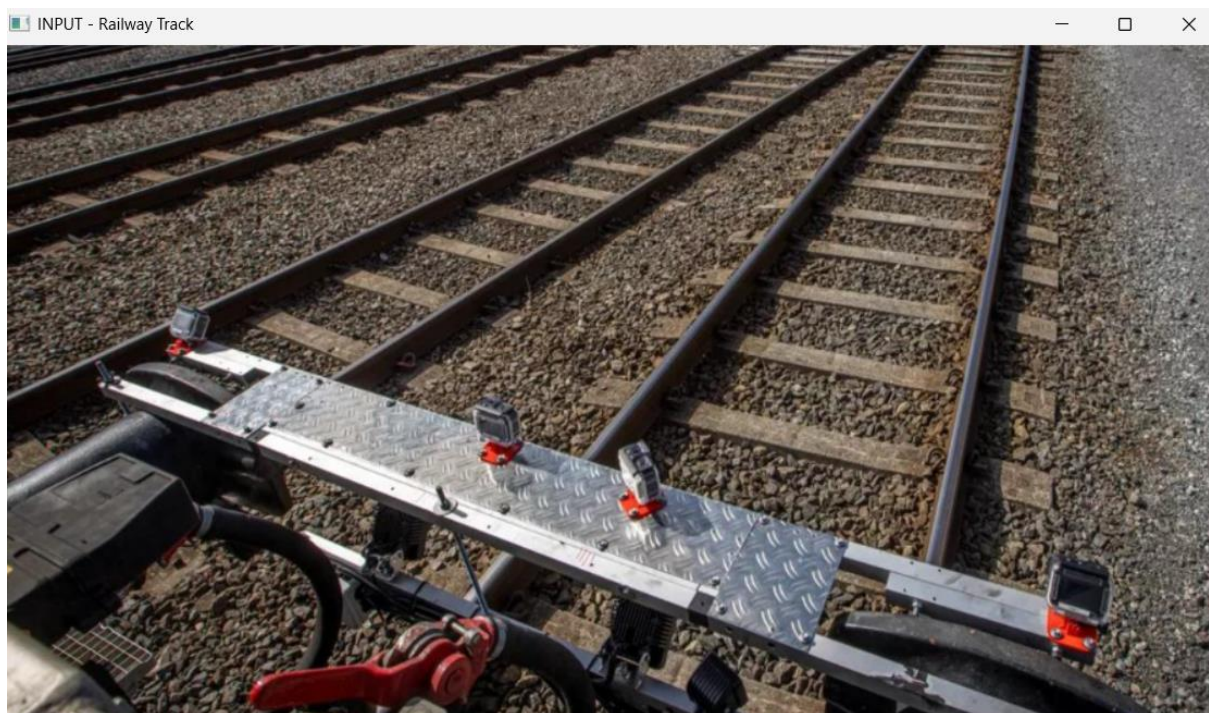


Fig 4.1 – Input Image – Original Railway Track:

Canny Edge Detection: Highlights the important edges of the railway tracks and surrounding objects.

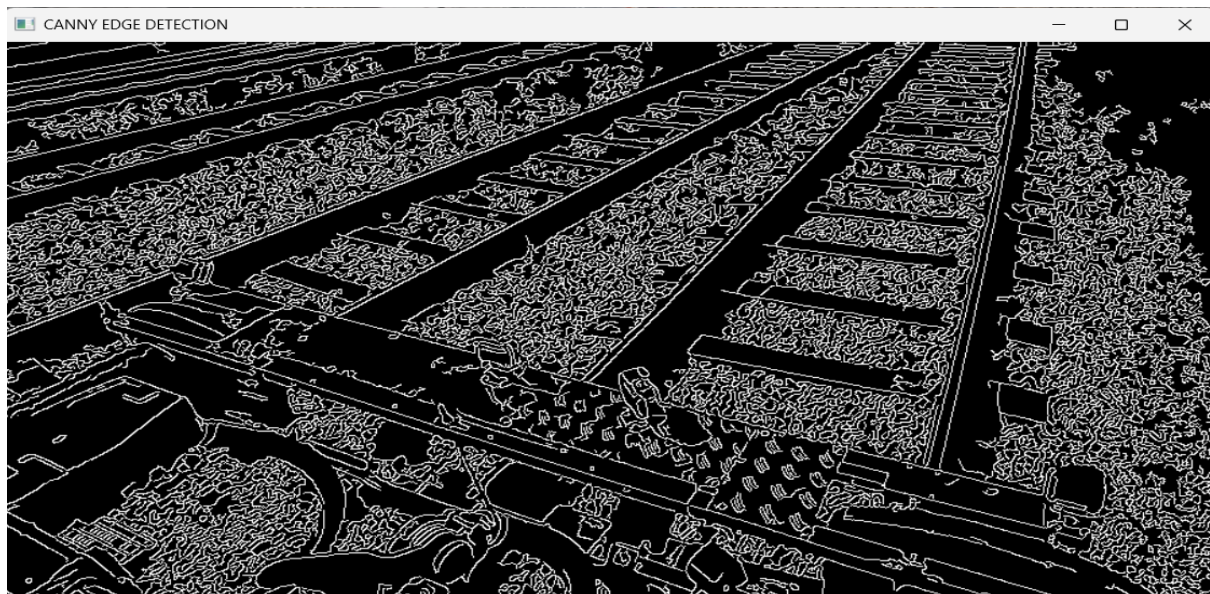


Fig 4.2 – Canny Edge Detection Result

Final Output: Displays the detected railway lines in green over the original image.

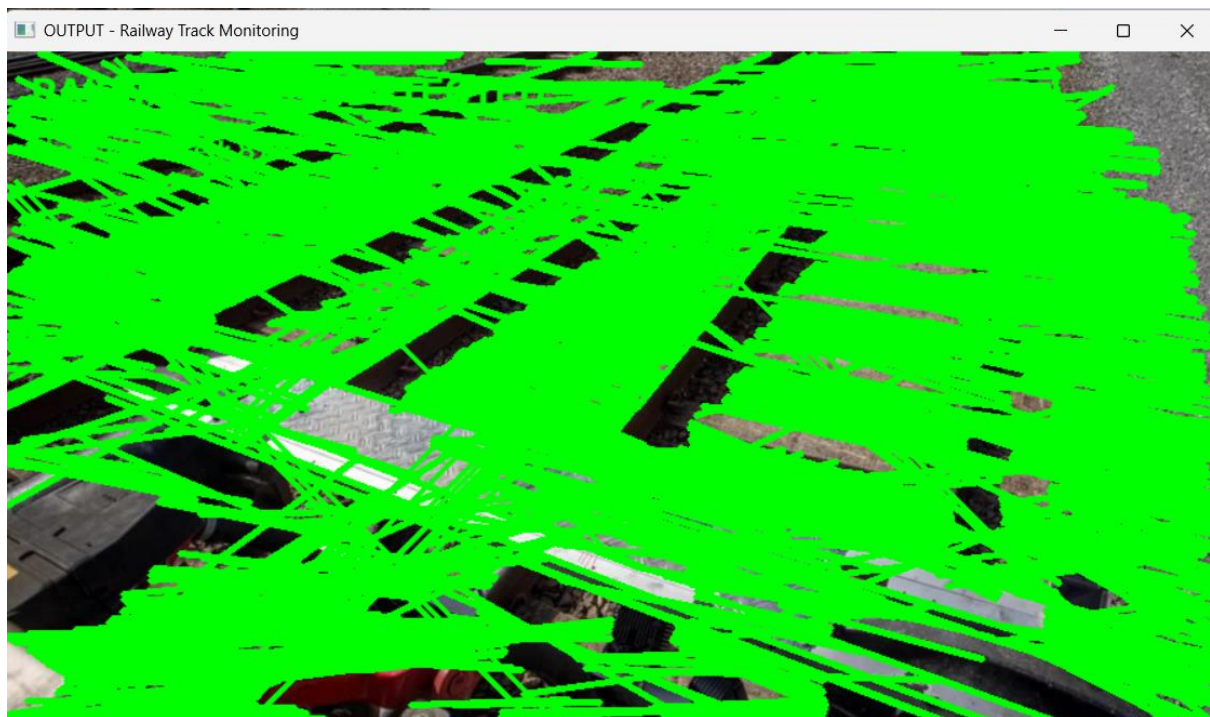


Fig 4.3 – Final Output – Railway Track Line Detection

Execution Result: Shows successful processing with **875 lines detected** using Canny + Hough Transform.

```
----- RESULT: C:/Users/HOSHITA/OneDrive/Desktop/CV ASS  
Image loaded successfully.  
-----  
RAILWAY TRACK MONITORING SYSTEM  
-----  
Input Image      : IMAGE.png  
Method           : Canny + Hough Transform  
Lines Detected   : 875  
Status           : Detection Completed  
-----
```

Fig4.4 – Program Execution Result – Railway Track Monitoring

5. CONCLUSION

The Real-Time Railway Track Monitoring System (RTMTS) with OpenCV illustrates how computer vision could effectively detect railway track lines in images. The integration of converting images to grayscale, performing Gaussian filtering, applying Canny Edge Detection, and using Probabilistic Hough Line Transform helped in identifying and marking the railway track lines. Consequently, the RTMTS successfully found 875 lines from the image, thus indicating efficacy in the image-processing pipeline used in railway track localization.

Although the Canny and Hough-based technique is computationally efficient and serves well in basic track detection, limitations arise when trying to incorporate it in complex railway scenarios, lighting changes, and actual track failures. The project proposes possible integration of the YOLO-based object detection into the system for more effective and reliable safety-critical system. YOLO leads to improved detection results while maintaining real-time performance.